ASSIGNMENT -2

COMPUTER NETWORKS
(PCCST501)

Submitted By: Emin Benny

Submitted To: Ms. Arsha J K

Class: S5 CSE A

Roll No: 28

SOCKET PROGRAMMING PRACTICE ASSIGNMENTS

Question 1 – Hello Client-Server (Beginner)

Problem Statement

Develop a TCP client-server application where:

• The client sends the message “Hello Server”.

• The server receives the message and displays it.

• The server replies with “Hello Client”.

• The client displays the received response.

---

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080

int main()
{
   int serverSocket, clientSocket;
   struct sockaddr_in serverAddr;
   socklen_t addrSize;
   char buffer[1024];

   /* Create Socket */

   serverSocket = socket(AF_INET, SOCK_STREAM, 0);
   if(serverSocket < 0)
   {
      perror("Socket Creation Failed");
      exit(1);
   }
   printf("Socket Created Successfully...\n");

   /* Server Address */

   serverAddr.sin_family = AF_INET;
   serverAddr.sin_addr.s_addr = INADDR_ANY;
   serverAddr.sin_port = htons(PORT);
```

```
/* Bind */

if(bind(serverSocket,
        (struct sockaddr*)&serverAddr,
        sizeof(serverAddr)) < 0)
{
   perror("Bind Failed");
   exit(1);
}
printf("Bind Successful...\n");

/* Listen */

listen(serverSocket,5);
printf("Waiting for Client...\n");

addrSize = sizeof(serverAddr);

/* Accept */

clientSocket = accept(serverSocket,
                      NULL,
                      NULL);
if(clientSocket < 0)
{
   perror("Accept Failed");
   exit(1);
}
printf("Client Connected...\n");

/* Receive */

recv(clientSocket,
     buffer,
     sizeof(buffer),
     0);
printf("Client Says : %s\n",buffer);

/* Reply */

strcpy(buffer,"Hello Client");
send(clientSocket,
     buffer,
     strlen(buffer)+1,
     0);
printf("Reply Sent...\n");
```

```
    close(clientSocket);
    close(serverSocket);

    printf("Connection Closed...\n");

    return 0;
}
```

client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080

int main() {

    int sockfd;
    struct sockaddr_in serverAddr;
    char buffer[1024];

    /* Create Socket */

    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if(sockfd < 0)
    {
        perror("Socket Creation Failed");
        exit(1);
    }
    printf("Socket Created Successfully...\n");

    /* Server Address */

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    inet_pton(AF_INET,
            "127.0.0.1",
            &serverAddr.sin_addr);

    /* Connect */

    if(connect(sockfd,
             (struct sockaddr *)&serverAddr,
```

```
            sizeof(serverAddr)) < 0)
    {
       perror("Connection Failed");
       exit(1);
    }
    printf("Connected to Server...\n");

    /* Read Message */

    printf("Enter Message : ");
    fgets(buffer,sizeof(buffer),stdin);

    /* Send */

    send(sockfd,
         buffer,
         strlen(buffer)+1,
         0);
    printf("Message Sent...\n");

    /* Receive */

    recv(sockfd,
         buffer,
         sizeof(buffer),
         0);

    printf("\nReply from Server : %s\n",
           buffer);

    /* Close */

    close(sockfd);

    printf("Connection Closed...\n");

    return 0;
}
```

SCREENSHOTS

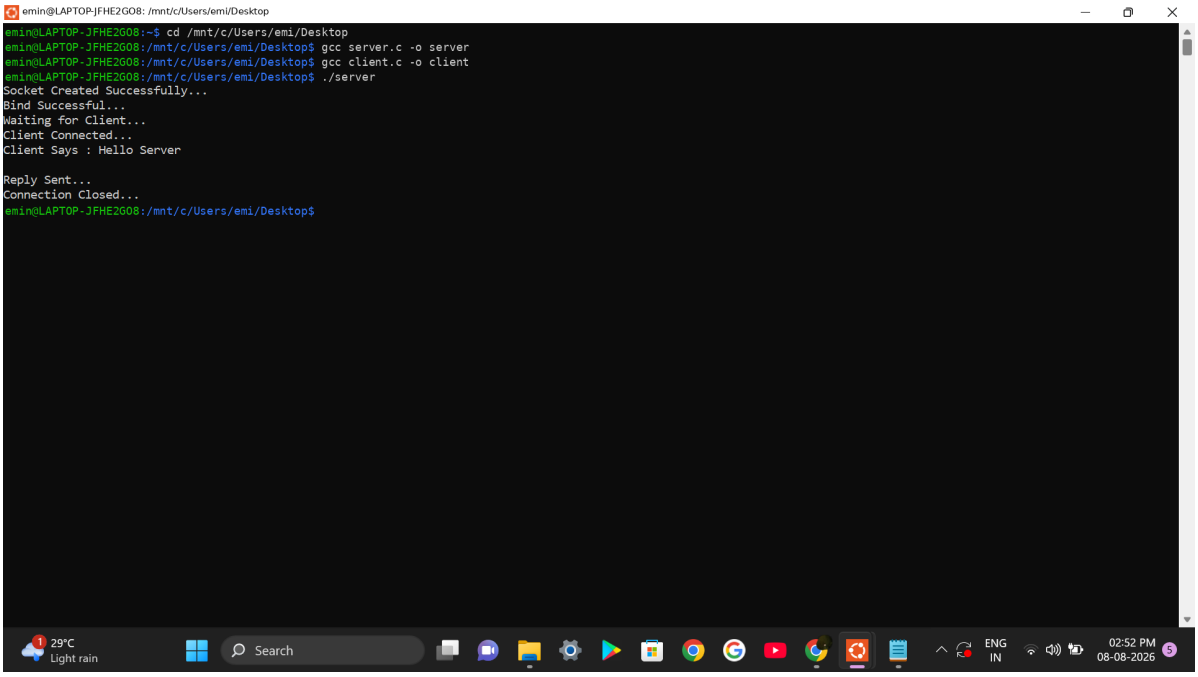


Fig 1.1: Server Terminal

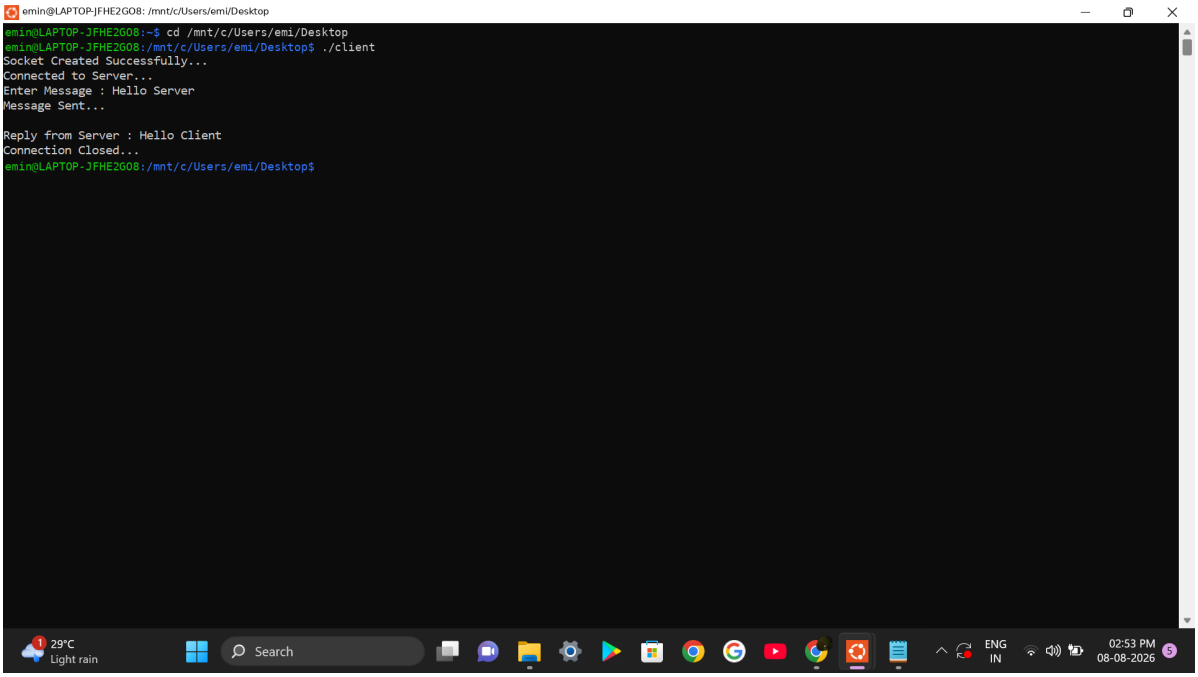


Fig 1.2: Client Terminal

Question 2 – Arithmetic Server

Problem Statement

Develop a TCP client-server application where:

• The client inputs two integers.

• The client sends both numbers to the server.

• The server calculates:
– Addition
– Subtraction
– Multiplication
– Division
• The server sends all results back.
• The client displays the received results.

---

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080

int main()
{
   int serverSocket, clientSocket;
   struct sockaddr_in serverAddr;
   int a, b;
   int addition, subtraction, multiplication;
   float division;

   serverSocket = socket(AF_INET, SOCK_STREAM, 0);
   if(serverSocket < 0)
   {
      perror("Socket Creation Failed");
      exit(1);
   }
   printf("Socket Created Successfully...\n");

   serverAddr.sin_family = AF_INET;
   serverAddr.sin_addr.s_addr = INADDR_ANY;
   serverAddr.sin_port = htons(PORT);

   if(bind(serverSocket,
         (struct sockaddr*)&serverAddr,
         sizeof(serverAddr)) < 0)
   {
      perror("Bind Failed");
      exit(1);
   }
   printf("Bind Successful...\n");
```

```
    listen(serverSocket, 5);
    printf("Waiting for Client...\n");

    clientSocket = accept(serverSocket, NULL, NULL);
    if(clientSocket < 0)
    {
        perror("Accept Failed");
        exit(1);
    }
    printf("Client Connected...\n");

    recv(clientSocket, &a, sizeof(a), 0);
    recv(clientSocket, &b, sizeof(b), 0);
    printf("Received Numbers: %d and %d\n", a, b);

    addition = a + b;
    subtraction = a - b;
    multiplication = a * b;
    if(b != 0)
        division = (float)a / b;
    else
        division = 0;

    send(clientSocket, &addition, sizeof(addition), 0);
    send(clientSocket, &subtraction, sizeof(subtraction), 0);
    send(clientSocket, &multiplication, sizeof(multiplication), 0);
    send(clientSocket, &division, sizeof(division), 0);
    printf("Results Sent...\n");

    close(clientSocket);
    close(serverSocket);

    printf("Connection Closed...\n");

    return 0;
}
```

client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
```

```
int main()
{
   int sockfd;
   struct sockaddr_in serverAddr;
   int a, b;
   int addition, subtraction, multiplication;
   float division;

   sockfd = socket(AF_INET, SOCK_STREAM, 0);
   if(sockfd < 0)
   {
      perror("Socket Creation Failed");
      exit(1);
   }
   printf("Socket Created Successfully...\n");

   serverAddr.sin_family = AF_INET;
   serverAddr.sin_port = htons(PORT);
   inet_pton(AF_INET, "127.0.0.1", &serverAddr.sin_addr);

   if(connect(sockfd,
            (struct sockaddr *)&serverAddr,
            sizeof(serverAddr)) < 0)
   {
      perror("Connection Failed");
      exit(1);
   }
   printf("Connected to Server...\n");

   printf("Enter first number: ");
   scanf("%d", &a);
   printf("Enter second number: ");
   scanf("%d", &b);

   send(sockfd, &a, sizeof(a), 0);
   send(sockfd, &b, sizeof(b), 0);

   recv(sockfd, &addition, sizeof(addition), 0);
   recv(sockfd, &subtraction, sizeof(subtraction), 0);
   recv(sockfd, &multiplication, sizeof(multiplication), 0);
   recv(sockfd, &division, sizeof(division), 0);

   printf("\nAddition : %d\n", addition);
   printf("Subtraction : %d\n", subtraction);
   printf("Multiplication : %d\n", multiplication);
   printf("Division : %.2f\n", division);
```

```
    close(sockfd);

    printf("Connection Closed...\n");

    return 0;
}
```

SCREENSHOTS

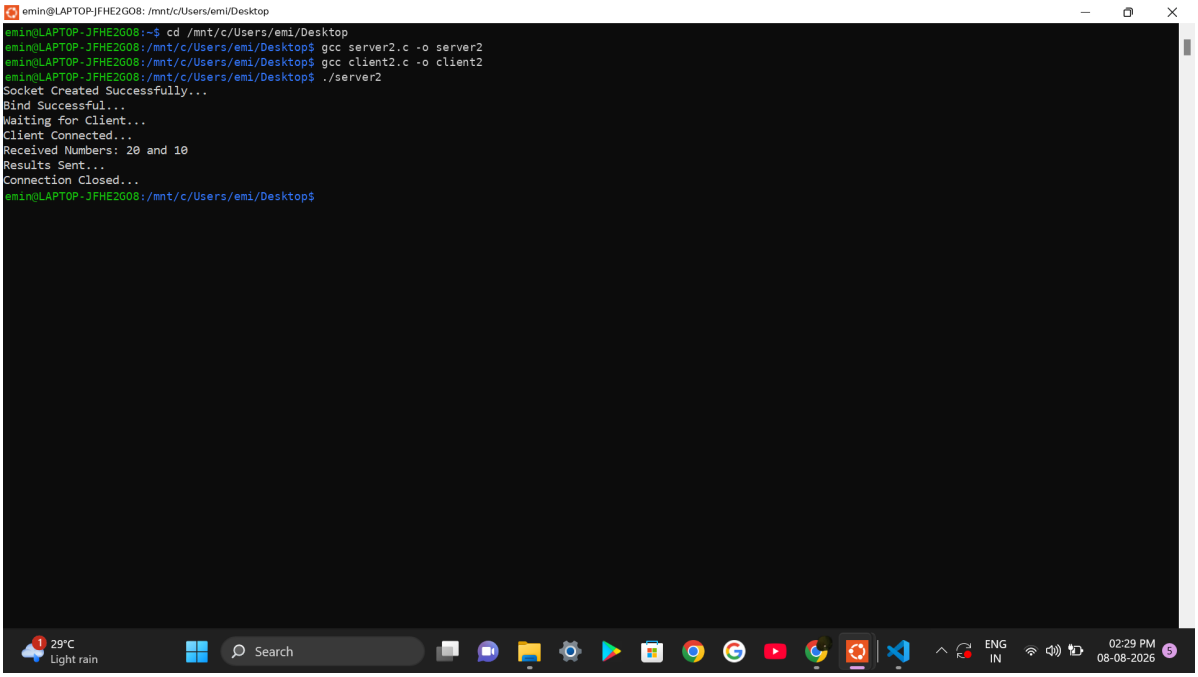


Fig 2.1: Server Terminal

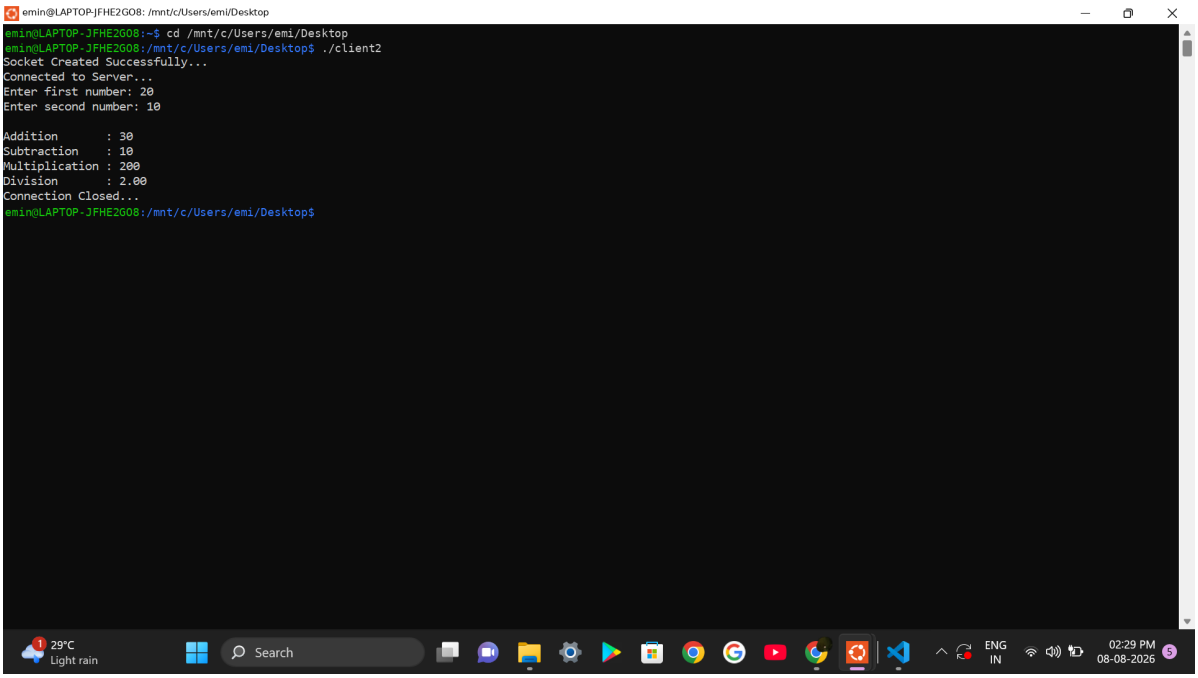


Fig 2.2: Client Terminal

Question 3 – String Processing Server

Problem Statement

Develop a TCP client-server application where:

• The client inputs a string.

• The server performs the following operations:

– Find the string length

– Convert the string to uppercase

– Reverse the string

• The server sends the processed results.

• The client displays all outputs.

---

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080

int main()
{
   int serverSocket, clientSocket;
   struct sockaddr_in serverAddr;
   char buffer[1024];
   char uppercase[1024];
   char reverse[1024];
   int length;

   serverSocket = socket(AF_INET, SOCK_STREAM, 0);
   if(serverSocket < 0)
   {
      perror("Socket Creation Failed");
      exit(1);
   }
   printf("Socket Created Successfully...\n");

   serverAddr.sin_family = AF_INET;
```

```
serverAddr.sin_addr.s_addr = INADDR_ANY;
serverAddr.sin_port = htons(PORT);

if(bind(serverSocket,
        (struct sockaddr*)&serverAddr,
        sizeof(serverAddr)) < 0)
{
   perror("Bind Failed");
   exit(1);
}
printf("Bind Successful...\n");

listen(serverSocket, 5);
printf("Waiting for Client...\n");

clientSocket = accept(serverSocket, NULL, NULL);
if(clientSocket < 0)
{
   perror("Accept Failed");
   exit(1);
}
printf("Client Connected...\n");

recv(clientSocket, buffer, sizeof(buffer), 0);
buffer[strcspn(buffer, "\n")] = '\0';
printf("Received String: %s\n", buffer);

/* Find length */
length = strlen(buffer);

/* Convert to uppercase */
strcpy(uppercase, buffer);
for(int i = 0; uppercase[i] != '\0'; i++)
{
   uppercase[i] = toupper(uppercase[i]);
}

/* Reverse string */
for(int i = 0; i < length; i++)
{
   reverse[i] = buffer[length - 1 - i];
}
reverse[length] = '\0';

/* Send results */
send(clientSocket, &length, sizeof(length), 0);
send(clientSocket, uppercase, sizeof(uppercase), 0);
```

```
    send(clientSocket, reverse, sizeof(reverse), 0);
    printf("Results Sent...\n");

    close(clientSocket);
    close(serverSocket);

    printf("Connection Closed...\n");

    return 0;
}
```

client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080

int main()
{
    int sockfd;
    struct sockaddr_in serverAddr;
    char buffer[1024];
    char uppercase[1024];
    char reverse[1024];
    int length;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if(sockfd < 0)
    {
        perror("Socket Creation Failed");
        exit(1);
    }
    printf("Socket Created Successfully...\n");

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    inet_pton(AF_INET, "127.0.0.1", &serverAddr.sin_addr);

    if(connect(sockfd,
            (struct sockaddr *)&serverAddr,
            sizeof(serverAddr)) < 0)
    {
```

```
        perror("Connection Failed");
        exit(1);
    }
    printf("Connected to Server...\n");

    printf("Enter a string: ");
    fgets(buffer, sizeof(buffer), stdin);

    /* Send string */
    send(sockfd, buffer, sizeof(buffer), 0);

    /* Receive results */
    recv(sockfd, &length, sizeof(length), 0);
    recv(sockfd, uppercase, sizeof(uppercase), 0);
    recv(sockfd, reverse, sizeof(reverse), 0);

    printf("\nString Length : %d\n", length);
    printf("Uppercase : %s\n", uppercase);
    printf("Reverse : %s\n", reverse);

    close(sockfd);

    printf("Connection Closed...\n");

    return 0;
}
```

SCREENSHOTS

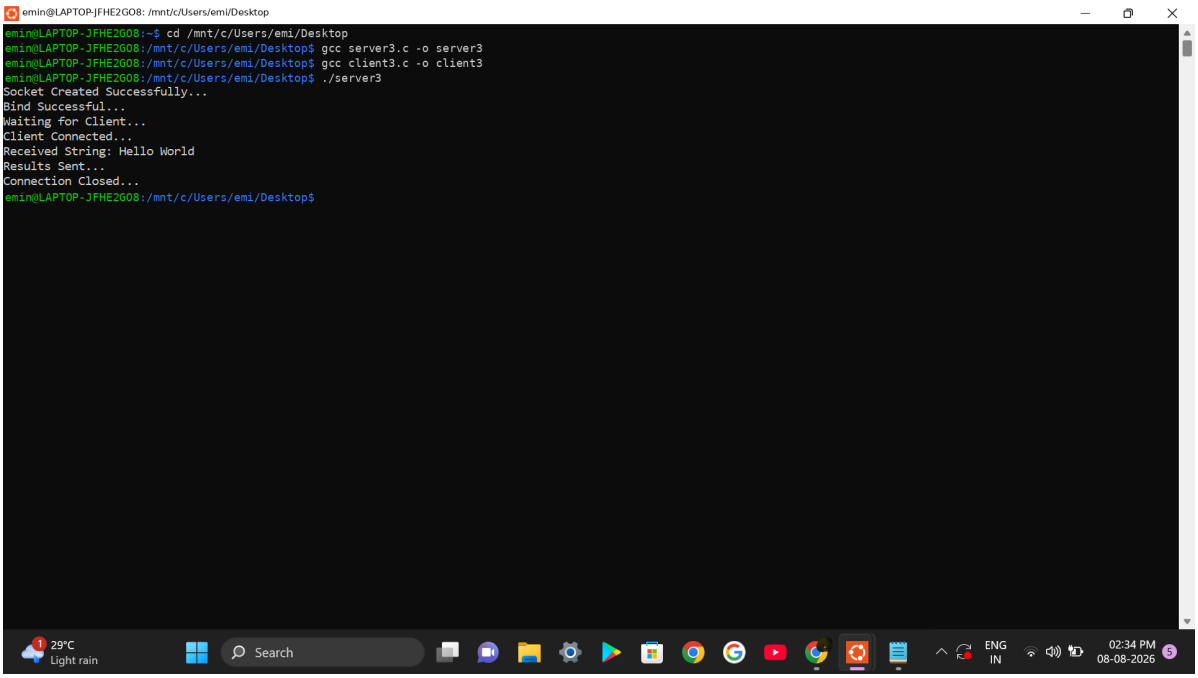


Fig 3.1: Server Terminal

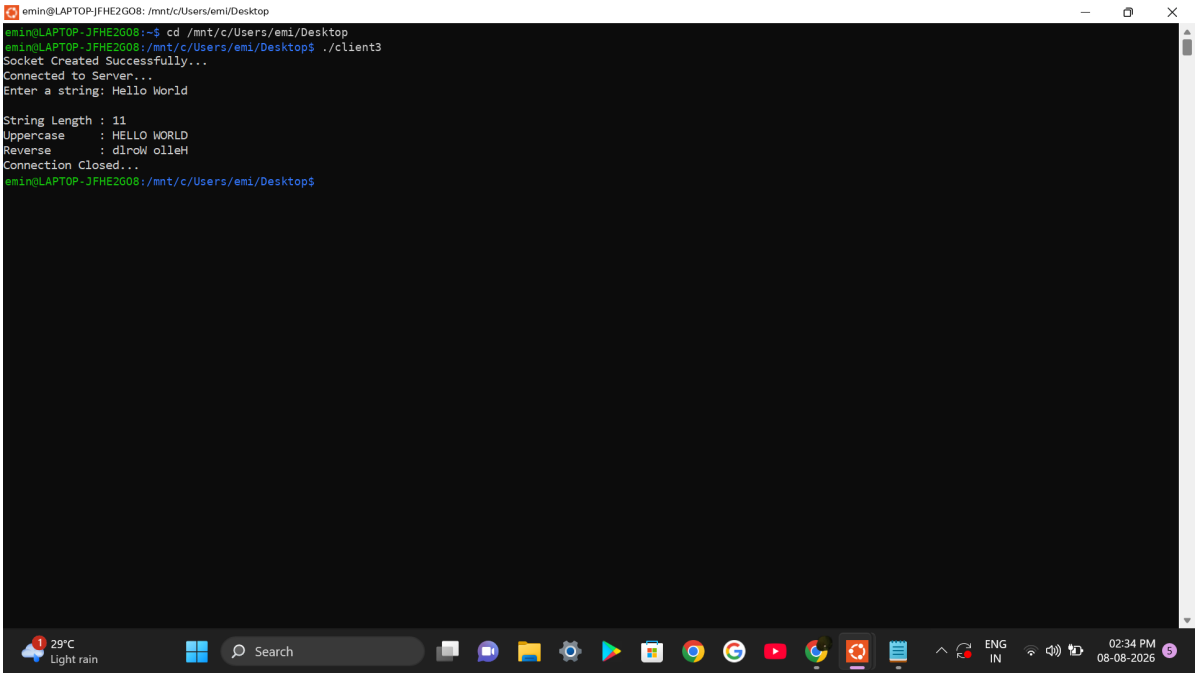


Fig 3.2: Client Terminal

Question 4 – Array Processing Server

Problem Statement

Develop a TCP client-server application where:

• The client inputs N integers.

• The array is sent to the server.

• The server computes:

– Sum

– Average

– Maximum Element

– Minimum Element

• The server sends the results.

• The client displays them.

---

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
```

```
#define PORT 8080
#define MAX 100

int main()
{
   int serverSocket, clientSocket;
   struct sockaddr_in serverAddr;
   int n;
   int arr[MAX];
   int sum = 0;
   int max, min;
   float average;

   serverSocket = socket(AF_INET, SOCK_STREAM, 0);
   if(serverSocket < 0)
   {
      perror("Socket Creation Failed");
      exit(1);
   }
   printf("Socket Created Successfully...\n");

   serverAddr.sin_family = AF_INET;
   serverAddr.sin_addr.s_addr = INADDR_ANY;
   serverAddr.sin_port = htons(PORT);

   if(bind(serverSocket,
         (struct sockaddr*)&serverAddr,
         sizeof(serverAddr)) < 0)
   {
      perror("Bind Failed");
      exit(1);
   }
   printf("Bind Successful...\n");

   listen(serverSocket, 5);
   printf("Waiting for Client...\n");

   clientSocket = accept(serverSocket, NULL, NULL);
   if(clientSocket < 0)
   {
      perror("Accept Failed");
      exit(1);
   }
   printf("Client Connected...\n");

   /* Receive N */
   recv(clientSocket, &n, sizeof(n), 0);
```

```
    /* Receive array */
    recv(clientSocket, arr, sizeof(arr), 0);
    printf("Received %d integers.\n", n);

    /* Calculate sum */
    for(int i = 0; i < n; i++)
    {
        sum = sum + arr[i];
    }

    /* Find maximum and minimum */
    max = arr[0];
    min = arr[0];
    for(int i = 1; i < n; i++)
    {
        if(arr[i] > max)
            max = arr[i];
        if(arr[i] < min)
            min = arr[i];
    }

    /* Calculate average */
    average = (float)sum / n;

    /* Send results */
    send(clientSocket, &sum, sizeof(sum), 0);
    send(clientSocket, &average, sizeof(average), 0);
    send(clientSocket, &max, sizeof(max), 0);
    send(clientSocket, &min, sizeof(min), 0);
    printf("Results Sent...\n");

    close(clientSocket);
    close(serverSocket);

    printf("Connection Closed...\n");

    return 0;
}
```

client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
```

```
#define PORT 8080
#define MAX 100

int main()
{
   int sockfd;
   struct sockaddr_in serverAddr;
   int n;
   int arr[MAX];
   int sum;
   int max;
   int min;
   float average;

   sockfd = socket(AF_INET, SOCK_STREAM, 0);
   if(sockfd < 0)
   {
      perror("Socket Creation Failed");
      exit(1);
   }
   printf("Socket Created Successfully...\n");

   serverAddr.sin_family = AF_INET;
   serverAddr.sin_port = htons(PORT);
   inet_pton(AF_INET, "127.0.0.1", &serverAddr.sin_addr);

   if(connect(sockfd,
            (struct sockaddr *)&serverAddr,
            sizeof(serverAddr)) < 0)
   {
      perror("Connection Failed");
      exit(1);
   }
   printf("Connected to Server...\n");

   printf("Enter number of integers: ");
   scanf("%d", &n);

   printf("Enter %d integers:\n", n);
   for(int i = 0; i < n; i++)
   {
      scanf("%d", &arr[i]);
   }

   /* Send N */
   send(sockfd, &n, sizeof(n), 0);
```

```
    /* Send array */
    send(sockfd, arr, sizeof(arr), 0);

    /* Receive results */
    recv(sockfd, &sum, sizeof(sum), 0);
    recv(sockfd, &average, sizeof(average), 0);
    recv(sockfd, &max, sizeof(max), 0);
    recv(sockfd, &min, sizeof(min), 0);

    printf("\n----- Results -----\n");
    printf("Sum : %d\n", sum);
    printf("Average : %.2f\n", average);
    printf("Maximum Element : %d\n", max);
    printf("Minimum Element : %d\n", min);

    close(sockfd);

    printf("Connection Closed...\n");

    return 0;
}
```

SCREENSHOTS

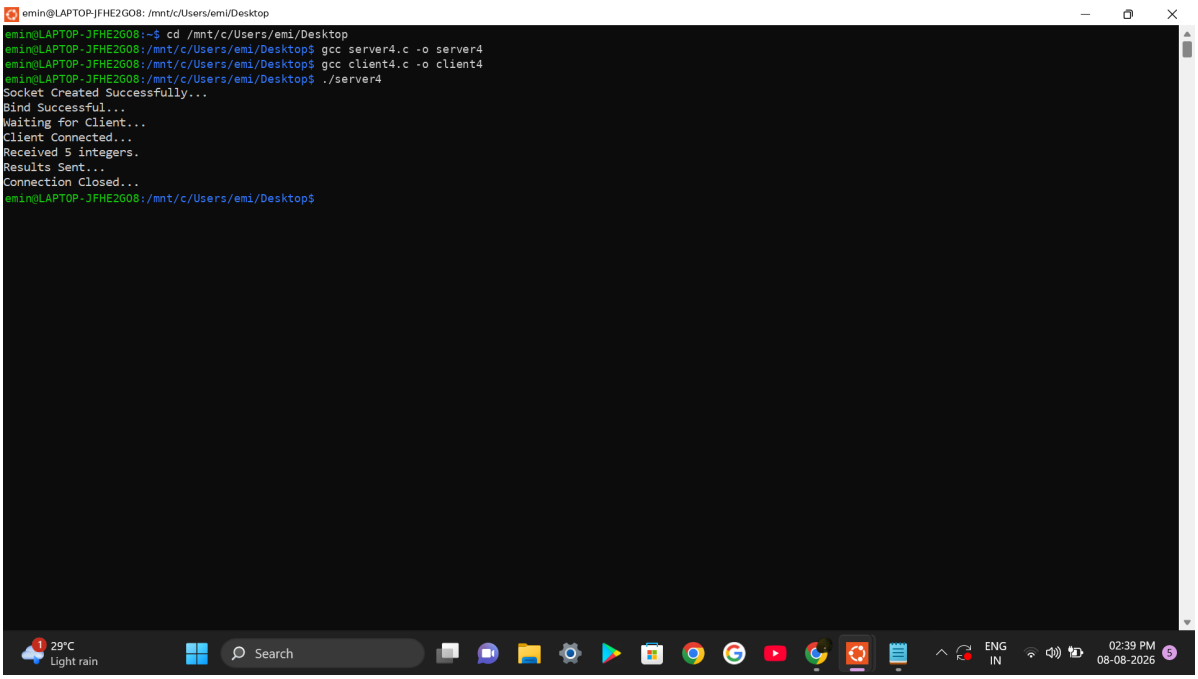


Fig 4.1: Server Terminal

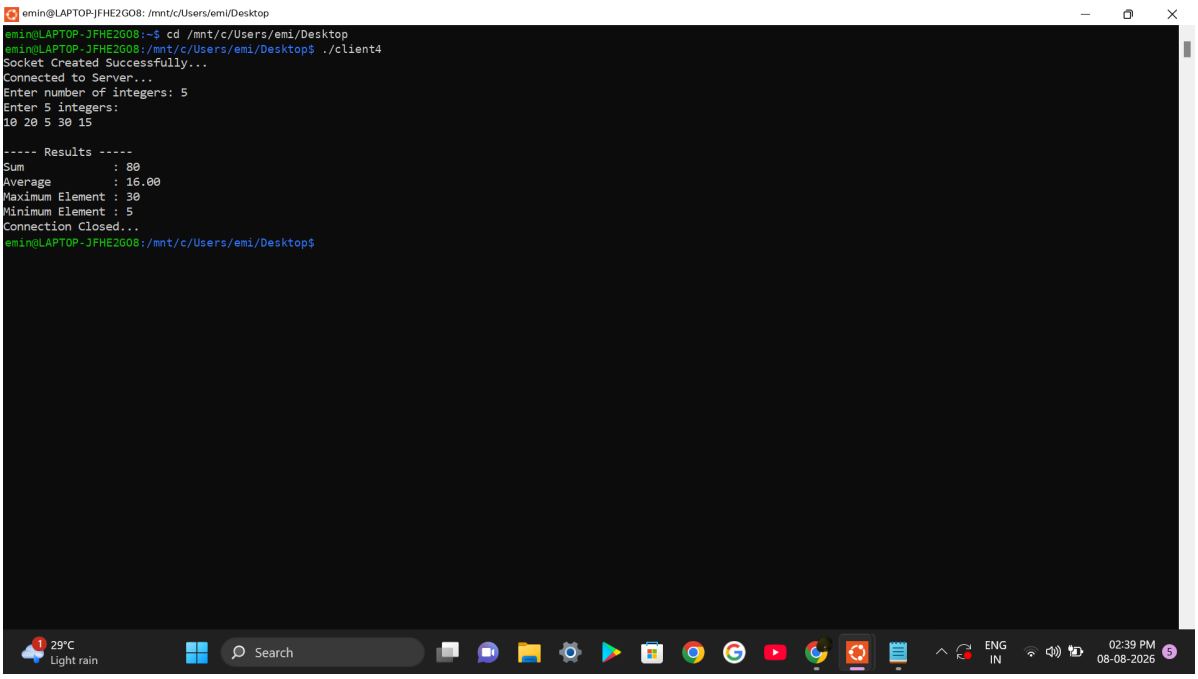


Fig 4.2: Client Terminal

Question 5 – Matrix Analyzer

Problem Statement

Develop a TCP client-server application where:

• The client inputs a square matrix of order N.

• The matrix is sent to the server.

• The server determines:

– Matrix Type

– Sum of Matrix Elements

– Trace of Matrix

• The server sends the results.

• The client displays the received information.

Matrix Types

• Diagonal Matrix

• Upper Triangular Matrix

• Lower Triangular Matrix

• General Matrix

---

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

```
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
#define MAX 100

int main()
{
   int serverSocket, clientSocket;
   struct sockaddr_in serverAddr;
   int n;
   int matrix[MAX][MAX];
   int sum = 0;
   int trace = 0;
   int diagonal = 1;
   int upper = 1;
   int lower = 1;
   char matrixType[30];

   serverSocket = socket(AF_INET, SOCK_STREAM, 0);
   if(serverSocket < 0)
   {
      perror("Socket Creation Failed");
      exit(1);
   }
   printf("Socket Created Successfully...\n");

   serverAddr.sin_family = AF_INET;
   serverAddr.sin_addr.s_addr = INADDR_ANY;
   serverAddr.sin_port = htons(PORT);

   if(bind(serverSocket,
         (struct sockaddr *)&serverAddr,
         sizeof(serverAddr)) < 0)
   {
      perror("Bind Failed");
      exit(1);
   }
   printf("Bind Successful...\n");

   listen(serverSocket, 5);
   printf("Waiting for Client...\n");

   clientSocket = accept(serverSocket, NULL, NULL);
   if(clientSocket < 0)
   {
      perror("Accept Failed");
```

```
    exit(1);
}
printf("Client Connected...\n");

/* Receive N */
recv(clientSocket, &n, sizeof(n), 0);

/* Receive matrix */
recv(clientSocket, matrix, sizeof(matrix), 0);
printf("Received %d x %d matrix.\n", n, n);

/* Calculate sum and trace */
for(int i = 0; i < n; i++)
{
    for(int j = 0; j < n; j++)
    {
        sum = sum + matrix[i][j];
        if(i == j)
        {
            trace = trace + matrix[i][j];
        }
    }
}

/* Check matrix type */
for(int i = 0; i < n; i++)
{
    for(int j = 0; j < n; j++)
    {
        /* Diagonal Matrix */
        if(i != j && matrix[i][j] != 0)
        {
            diagonal = 0;
        }
        /* Upper Triangular Matrix */
        if(i > j && matrix[i][j] != 0)
        {
            upper = 0;
        }
        /* Lower Triangular Matrix */
        if(i < j && matrix[i][j] != 0)
        {
            lower = 0;
        }
    }
}
```

```
    /* Determine matrix type */
    if(diagonal == 1)
    {
        sprintf(matrixType, "Diagonal Matrix");
    }
    else if(upper == 1)
    {
        sprintf(matrixType, "Upper Triangular Matrix");
    }
    else if(lower == 1)
    {
        sprintf(matrixType, "Lower Triangular Matrix");
    }
    else
    {
        sprintf(matrixType, "General Matrix");
    }

    /* Send results */
    send(clientSocket, matrixType, sizeof(matrixType), 0);
    send(clientSocket, &sum, sizeof(sum), 0);
    send(clientSocket, &trace, sizeof(trace), 0);
    printf("Results Sent...\n");

    close(clientSocket);
    close(serverSocket);

    printf("Connection Closed...\n");

    return 0;
}
```

client.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080
#define MAX 100

int main()
{
    int sockfd;
    struct sockaddr_in serverAddr;
```

```
int n;
int matrix[MAX][MAX];
int sum;
int trace;
char matrixType[30];

sockfd = socket(AF_INET, SOCK_STREAM, 0);
if(sockfd < 0)
{
   perror("Socket Creation Failed");
   exit(1);
}
printf("Socket Created Successfully...\n");

serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT);
inet_pton(AF_INET, "127.0.0.1", &serverAddr.sin_addr);

if(connect(sockfd,
         (struct sockaddr *)&serverAddr,
         sizeof(serverAddr)) < 0)
{
   perror("Connection Failed");
   exit(1);
}
printf("Connected to Server...\n");

printf("Enter order of matrix: ");
scanf("%d", &n);

printf("Enter %d x %d matrix elements:\n", n, n);
for(int i = 0; i < n; i++)
{
   for(int j = 0; j < n; j++)
   {
      scanf("%d", &matrix[i][j]);
   }
}

/* Send N */
send(sockfd, &n, sizeof(n), 0);

/* Send matrix */
send(sockfd, matrix, sizeof(matrix), 0);

/* Receive results */
recv(sockfd, matrixType, sizeof(matrixType), 0);
```

```
    recv(sockfd, &sum, sizeof(sum), 0);
    recv(sockfd, &trace, sizeof(trace), 0);

    printf("\n----- Results -----\n");
    printf("Matrix Type : %s\n", matrixType);
    printf("Sum of Elements : %d\n", sum);
    printf("Trace : %d\n", trace);

    close(sockfd);

    printf("Connection Closed...\n");

    return 0;
}
```

SCREENSHOTS

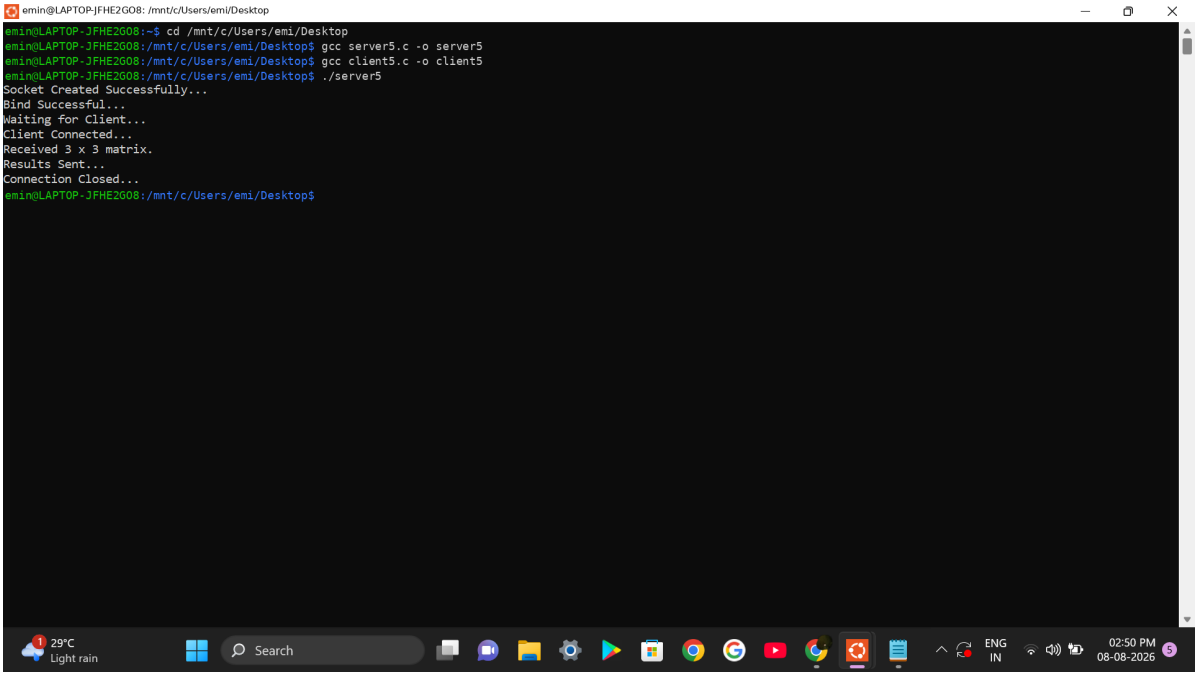


Fig 5.1: Server Terminal

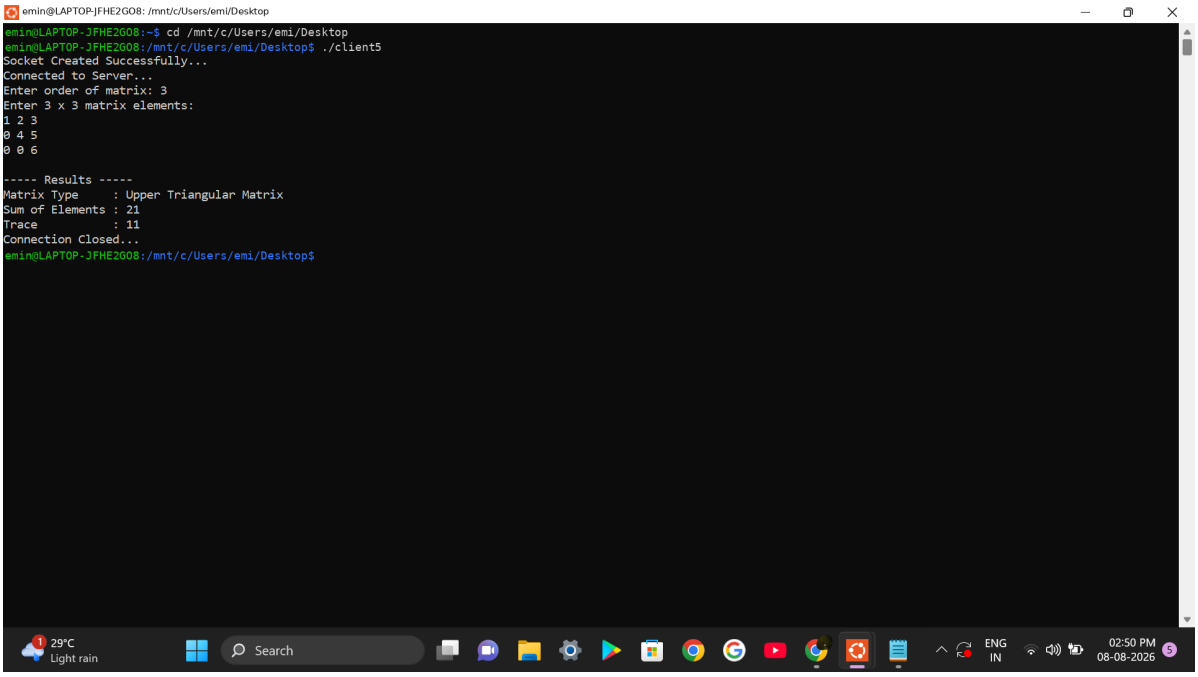


Fig 5.2: Client Terminal